Analyse der Implementierungsstrategie für CityFeedback

Die Grundidee ist solide: zwei klar getrennte Bounded Contexts, ein überschaubarer erster Schnitt (nur Beschwerdeerstellung) und TDD. Die folgenden Vorschläge zielen darauf, dass dieser Schnitt auch wirklich sauber bleibt und spätere Erweiterungen (Kommentare, Statuswechsel) nicht zu Umbauten führen.

Vorschlag 1: Aggregate mit Factory-Methode, Value Objects und eigener Identität

Problem im aktuellen Entwurf: Sieben lose Attribute (String title, String description, UUID id …) laden zu einem anämischen Modell ein: Ein Objekt mit Gettern/Settern, dessen Regeln irgendwo in Services liegen.

Vorschlag:

- **Erzeugung nur über eine statische Factory** (Complaint.create(...)), privater Konstruktor. Damit kann kein Complaint im ungültigen Zustand existieren. status = OPEN wird *im Aggregate* gesetzt, nicht vom Aufrufer übergeben.
- **Value Objects** statt Primitiven: ComplaintId, CitizenId, ComplaintTitle, ComplaintDescription. Category und Status bleiben Enums.
- **citizenId ist nur eine Referenz** auf den Identity Context (eigener Typ CitizenId, kein Import von Identity-Klassen). So bleibt die Kontexttrennung erhalten.
- **Zeit und ID von außen injizieren** (Clock, ComplaintIdGenerator bzw. Parameter), nicht Instant.now() oder UUID.randomUUID() direkt im Aggregate aufrufen.
- **Keine Setter**; Zustandsänderungen später als sprechende Methoden (close(), addComment()).

Begründung: Invarianten gehören ins Aggregate. Die Injektion von Zeit und ID macht Tests deterministisch.

Auswirkung: Etwas mehr Klassen am Anfang, dafür sind Tests einfach (Complaint.create(...) mit fester Clock), die Persistenz-Anbindung später ist ein reines Mapping-Thema, und Kommentare können als Teil des Aggregates oder als eigenes Aggregate nachgezogen werden, ohne die Basis zu ändern. Kommentare bewusst jetzt *nicht* modellieren (YAGNI), aber entscheiden und dokumentieren, ob sie innerhalb des Complaint-Aggregates liegen (Konsistenzgrenze) oder separat.

Vorschlag 2: Validierung in der Domäne, strukturiert und mit allen Fehlern auf einmal

Problem: Validierung wird oft nur im Controller/Service oder per Bean-Validation-Annotationen gemacht. Dann schützt sie das Aggregate nicht, und die Regeln sind verstreut.

Vorschlag:

- **Zwei Ebenen trennen:**
    1. *Domänenvalidierung* (Invarianten): Titel nicht leer/nur Whitespace, Längenlimit (z. B. 5–100 Zeichen), Beschreibung nicht leer, max. Länge (z. B. 2000), Kategorie nicht null. Sitzt in den Value Objects bzw. der Factory.

2. *Eingabeprüfung an der Anwendungsgrenze* (Format, Pflichtfelder im Request): bleibt dem späteren API-/Application-Layer überlassen und ist **nicht** Teil des ersten Schritts.

- **Eigene Exception** (InvalidComplaintException) mit **gesammelten Fehlern** (Feldname + Grund) statt „erste Fehlermeldung wirft". Das ist für Bürger deutlich nützlicher und gut testbar.
- **Normalisierung festlegen:** trim() vor der Prüfung? Zeichenlimit in Codepoints oder chars? Diese Entscheidungen früh treffen und als Test festhalten.
- **Grenzwerte als Konstanten** in der Domäne, damit Tests und Code dieselbe Quelle nutzen.
- **Kategorie:** Enum-Parsing ("roads" → ROADS) gehört an die Grenze, die Domäne bekommt nur ein gültiges Enum oder null (→ Fehler).

Begründung: Wer Invarianten nur außen prüft, kann sie über andere Einstiegspunkte (Import, Tests, andere Services) umgehen.

Auswirkung: Die Testliste wird präzise (Randwerte, Whitespace, null, Mehrfachfehler). Spätere REST-Fehlerantworten (HTTP 400 mit Feldfehlern) lassen sich direkt aus der Exception ableiten, ohne die Domäne zu ändern.

Vorschlag 3: Domain Event als unveränderlicher Fakt, vom Aggregate erzeugt, vom Application Service veröffentlicht

Problem: „Beim Erstellen wird BeschwerdeErstellt ausgelöst" lässt offen, *wer* auslöst und *wann*. Typische Fehler: Event wird vor dem erfolgreichen Speichern gefeuert, oder das Aggregate kennt einen Event-Bus.

Vorschlag:

- **Das Aggregate sammelt Events** (List<DomainEvent> intern, pullDomainEvents()), die Factory registriert ComplaintCreated. Das Aggregate kennt **keinen** Publisher.
- **Der Application Service** (CreateComplaintUseCase) speichert über das Repository und **veröffentlicht danach** die Events über einen Port (DomainEventPublisher).
- **Event als Java-record**, unveränderlich, mit minimalem, aber ausreichendem Inhalt: eventId, occurredAt, complaintId, citizenId, category. Keine Referenz auf das Aggregate selbst, keine Beschreibung/PII ohne Bedarf.
- **Namenskonvention:** Events beschreiben Vergangenes und sollten im Code konsistent sein. Fachsprache Deutsch vs. Code Englisch ist eine Entscheidung, die ihr einmal treffen solltet. Entweder durchgängig deutsch (BeschwerdeErstellt, Beschwerde) oder, wie hier vorgeschlagen, Englisch im Code (ComplaintCreated) mit einem Glossar, das den Bezug zur Fachsprache herstellt. Eine Mischung (Complaint + BeschwerdeErstellt) erzeugt dauerhaft Reibung.
- **Zuverlässigkeit:** Für den ersten Schritt reicht ein In-Memory-Publisher. Bewusst notieren, dass später ein Outbox-Pattern nötig werden kann, falls andere Kontexte das Event zuverlässig konsumieren sollen.

Begründung: Das Aggregate bleibt rein und testbar. Events entstehen nur bei erfolgreicher Erzeugung, und die Veröffentlichung nach dem Speichern verhindert „Phantom-Events".

Auswirkung: Tests können prüfen, dass genau ein Event mit korrekten Daten entsteht, ohne Infrastruktur. Später lassen sich Consumer (z. B. Benachrichtigung) anbinden, ohne das Aggregate anzufassen.

Vorschlag 4: Testbarkeit und TDD – Testpyramide, Testliste und Fakes statt Mocks

Problem: „Erst Tests, dann Code" reicht nicht, wenn die Tests an Implementierungsdetails kleben oder ohne Plan entstehen.

Vorschlag:

- **Testliste vor dem ersten Test** (Kent-Beck-Stil): Verhalten in kleinen Szenarien aufschreiben, z. B.
    - erzeugt Complaint mit Status OPEN
    - setzt createdAt aus der Clock
    - lehnt leeren/zu langen Titel ab
    - lehnt fehlende Kategorie ab
    - sammelt mehrere Validierungsfehler
    - registriert genau ein ComplaintCreated-Event
    - Use Case speichert und veröffentlicht erst danach
    - Use Case veröffentlicht nichts, wenn die Validierung scheitert
- **Red–Green–Refactor in kleinen Schritten**, ein Test pro Verhalten, Test-Namen als Spezifikation (@DisplayName auf Deutsch ist hier völlig in Ordnung).
- **Testebenen:**
    - *Unit-Tests* auf Domäne (schnell, kein Framework, kein Mockito nötig).
    - *Use-Case-Tests* mit **Fakes** (InMemoryComplaintRepository, RecordingEventPublisher, Clock.fixed(...)) statt Mock-Orgie. Fakes testen Verhalten, Mocks testen Aufrufe und brechen bei Refactorings.
    - Persistenz/REST/Integration **erst später**.
- **JUnit-5-Features nutzen:** @ParameterizedTest für Grenzwerte und ungültige Eingaben, assertAll für Mehrfachfehler.
- **Test-Data-Builder** (aComplaint().withTitle(...)) halten Tests lesbar.
- **Maven:** Surefire für Unit-Tests; Integrationstests später per Failsafe trennen. Optional JaCoCo, aber Verhalten ist wichtiger als Prozentzahlen.
- **Architektur absichern:** Optional ArchUnit-Test, der sicherstellt, dass domain keine Abhängigkeit auf Infrastruktur oder auf den Identity Context hat.

Begründung: Die Testliste verhindert, dass TDD zu „Tests nachträglich um fertigen Code herum" degeneriert. Fakes machen die Tests robust gegen Refactoring.

Auswirkung: Hohe Änderungssicherheit, schnelle Testläufe (Millisekunden), und die Tests dokumentieren die Fachregeln.

Vorschlag 5: Entwicklungsreihenfolge „inside-out“ mit vertikalem Schnitt

Empfohlene Reihenfolge:

1. **Ubiquitous Language klären** (Glossar, Sprache im Code, Validierungsgrenzen, Fragen wie: Darf jeder angemeldete Bürger Beschwerden erstellen?).
2. **Domäne:** Value Objects und Complaint.create(...) inkl. Validierung, Status, createdAt.
3. **Domain Event:** ComplaintCreated und Event-Sammlung im Aggregate.
4. **Ports definieren:** ComplaintRepository, DomainEventPublisher, CitizenId-Quelle (siehe unten).
5. **Use Case** CreateComplaint mit Command-Objekt, Fakes für Ports.
6. **Erst danach** Adapter: Persistenz (z. B. JPA/JDBC), REST-Controller, Event-Publisher-Implementierung, Integrationstests.

Zusätzlich beachten:

- **Identity Context nicht nachbauen.** Für den ersten Schritt reicht, dass der Use Case eine CitizenId als Eingabe erhält (aus dem authentifizierten Kontext). Ob der Bürger existiert/berechtigt ist, ist ein Anti-Corruption-Layer-Thema für später; vorerst als Annahme dokumentieren.
- **Nicht umsetzen, was nicht gebraucht wird:** Kommentare, Statuswechsel, Abfragen. Aber status als Enum sauber anlegen, damit spätere Übergänge (OPEN → IN_PROGRESS → CLOSED) möglich bleiben, ohne sie jetzt zu implementieren.
- **Maven früh sauber aufsetzen** (Java-Version, JUnit-5-BOM, Surefire), damit TDD vom ersten Commit an läuft.
- **Walking Skeleton als Ziel:** Nach Schritt 5 existiert ein vollständig getesteter Durchstich (Command → Aggregate → Repository → Event), an den Adapter später nur noch angedockt werden.

Begründung: Inside-out hält den Kern frei von Framework-Entscheidungen und liefert früh Feedback zu den wichtigsten Regeln.

Auswirkung: Technologieentscheidungen (Spring, JPA, Messaging) können spät und reversibel getroffen werden, und die Domäne bleibt portabel.

Zusammenfassung der Auswirkungen

Vorschlag	**Kurzfristiger Aufwand**	**Langfristiger Nutzen**
Aggregate mit Factory & Value Objects	mittel	Invarianten garantiert, einfache Erweiterung

Vorschlag	Kurzfristiger Aufwand	Langfristiger Nutzen
Validierung in der Domäne, Fehler sammeln	gering–mittel	Konsistente Regeln, gute Fehlermeldungen
Event vom Aggregate, Publish nach Save	mittel	Entkopplung, spätere Integration einfach
Testliste, Fakes, Test-Builder	gering	Schnelle, refactoring-feste Tests
Inside-out-Reihenfolge	gering	Späte, reversible Technologieentscheidungen